Johns Hopkins Engineering

Software Radio for Wireless Communications

Module 11 Live Lab

Dan Chew, Andrew Adams

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Goals:

- This Live Lab will:
  - Develop skills with Energy Detection.
  - Develop skills with Noise Estimation.

Outline

1. Questions from Last Week’s Homework
2. Background
3. Experiments

Last Week’s Homework

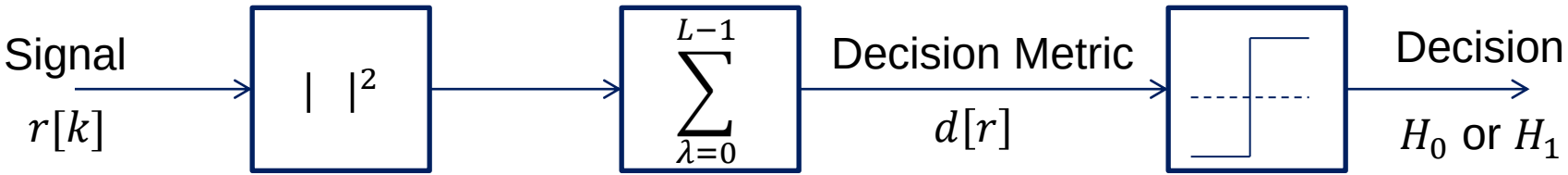
Last Week’s Homework

- Q & A

Background – Energy Detector

The Energy Detector

- The Energy Detector is the sum over some measurement length of the absolute-value-squared.



$$d[r] = E_r = \sum_{\lambda=0}^{L-1} |r[k-\lambda]|^2$$

$H_0: d[r] \leq \eta$ (no signal present)

$H_1: d[r] > \eta$ (signal present)

Theoretical Curves (1)

- For the energy detector, the probability of false alarm is implemented in Matlab as

```
gammainc( threshold/2/noisesigma^2, measurement_length, 'upper')
```

  - Note that this function is Normalized, and requires no Normalization step

- Matlab offers has the inverse gamma function.

Theoretical Curves (2)

- For the energy detector, the probability of detect is implemented in Matlab as

```
marcumq(sqrt(signal_amplitude^2*measurement_length)/noisesigma,sqrt(threshold)/noisesigma,measurement_length);
```

- Matlab does not offer the inverse of the Marcum Q function.

Threshold by Noise Estimate

- η should be a function of the local environment
  - Static values perform poorly
- How to set η in the presence of signal activity?
  - Quiet periods (time) never guaranteed
  - Quiet bands (frequency) never guaranteed

Energy Detection Experiments

Energy Detector in Noise, Length 1 (1)

- Let's put together a measurement matrix of noise.
  - Set up the parameters

```
measurement_length = 1;

number_of_measurements = 16384;
```

  - Create the noise

```
num_samples_in_simulation = measurement_length*number_of_measurements;

noisesigma = 1; % the standard deviation of one axis of noise
noise_signal = randn(measurement_length*number_of_measurements,1) + ...
1.0i*randn(measurement_length*number_of_measurements,1);
noise_signal = noise_signal - mean(noise_signal);
noise_signal = noise_signal/sqrt(var(noise_signal))*noisesigma*sqrt(2);

received_signal = noise_signal;
```

Energy Detector in Noise, Length 1 (2)

- Let's put together a measurement matrix of noise.
  - Reshape the received vector into a measurement matrix
  - Each column is a measurement

```
measurement_matrix = ...
reshape(received_signal, measurement_length, number_of_measurements);
```

  - Sum the absolute values of each element in each measurement

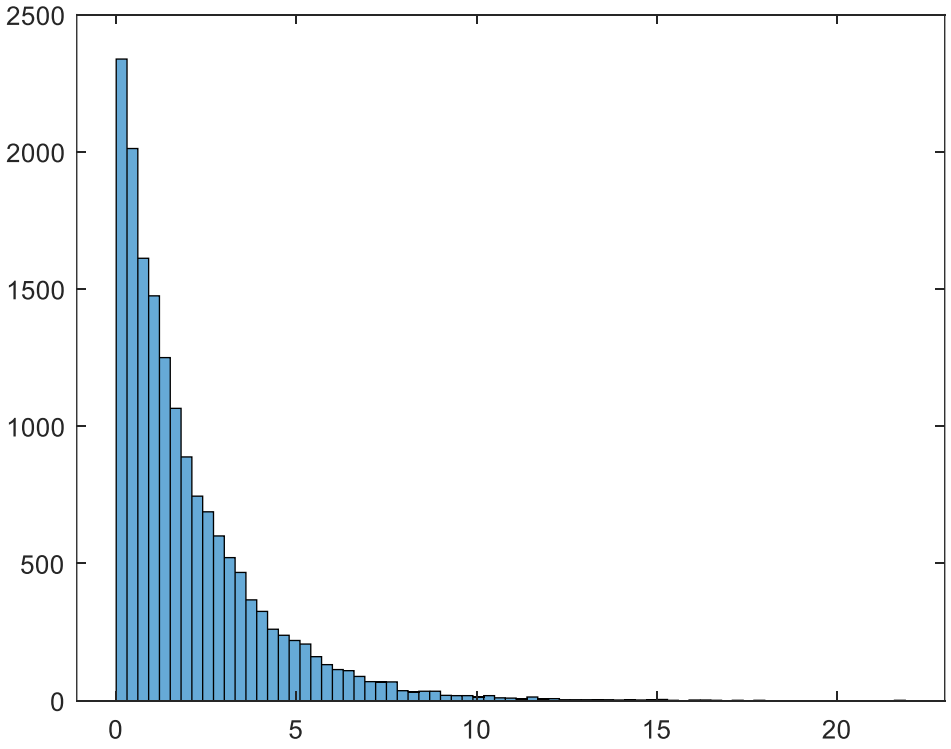
```
decision_metrics = sum(abs(measurement_matrix).^2, 1);
```

  - Plot the histogram

```
figure
histogram(decision_metrics)
```

Energy Detector in Noise, Length 1, Results

- You should have something like this.
- This is a chi-square distribution with two degrees of freedom.



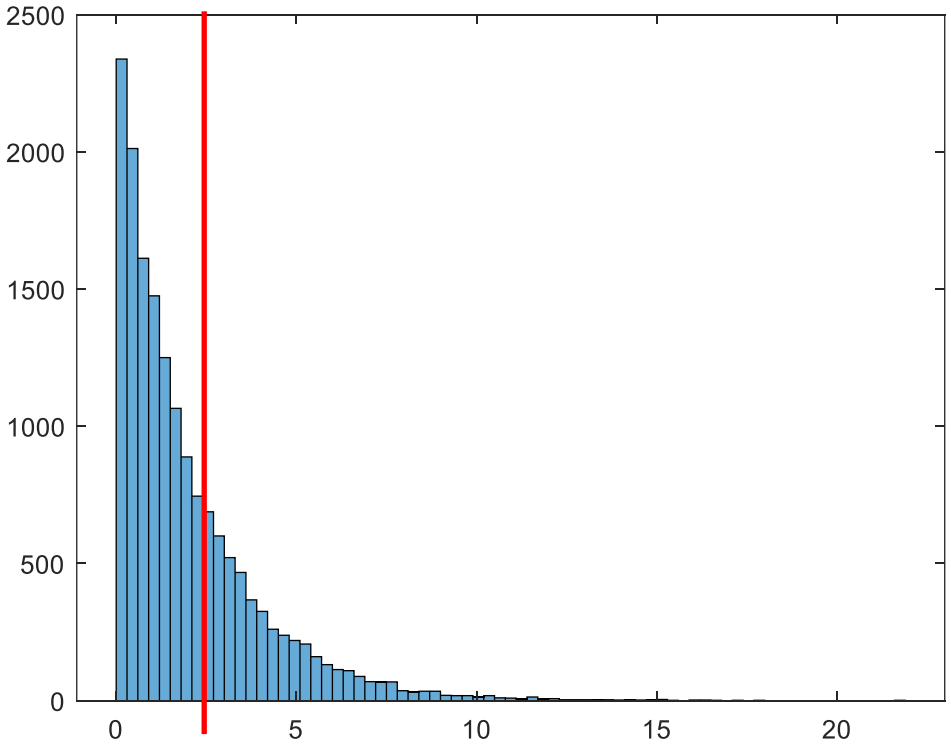
Energy Detector in Noise, Length 1, Threshold

- Now set a threshold at 2.5

```
threshold_value = 2.5;
detection_vector = decision_metrics > threshold_value;
probability_of_detection = detection_vector/length(detection_vector);
```

- Calculate probability of False Alarm

```
probability_of_false_alarm = sum(detection_vector)/length(detection_vector)

theoretical_probability_of_false_alarm = ...
   gammainc( threshold_value/2/noisesigma^2, measurement_length, 'upper')
```



What do you see?

Energy Detector in Noise, Length 64

- Adjust the measurement length from 1 to 64.
- What changes?
- What is the false alarm probability?
- Should anything else change?
  - If so, can we use anything we know, like the measurement length, to change it?

Energy Detector in Noise, sigma 10

- Set the measurement length back to 1.
- Set the threshold to 2.5.
- Adjust the noise sigma from 1 to 10.

- What is the false alarm probability?

- What can we change?
- How will we know the necessary values?

Noise Estimation Experiments

Experiment – Noiseless CW (1)

- Let's put together a sample matrix of a tone without noise.
  - Set up the parameters

```
num_samples_per_observation = 64;
num_observations = 256;
num_samples_in_simulation = num_samples_per_observation*num_observations;
```

  - Create the signal

```
test_signal = exp(1.0i*2*pi*0.125*[0:(num_samples_in_simulation-1)]);
test_signal=test_signal(:);
```

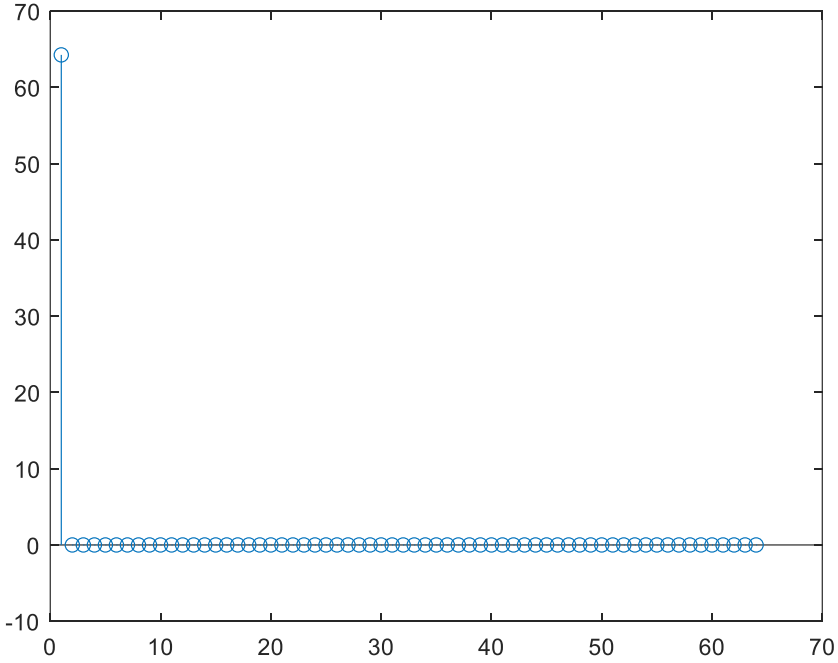
  - Reshape the vector

```
signal_matrix = reshape(test_signal, num_samples_per_observation,...
num_observations);

sigma_matrix = (signal_matrix*signal_matrix')/(num_observations-1);
```

Experiment – Noiseless CW (2)

- Now take the eigen values

```
eig_vector = eig(sigma_matrix);

sorted_eig_vector = sort(eig_vector, 'descend');

figure
stem(sorted_eig_vector)
```



Experiment – Noiseless Modulated (1)

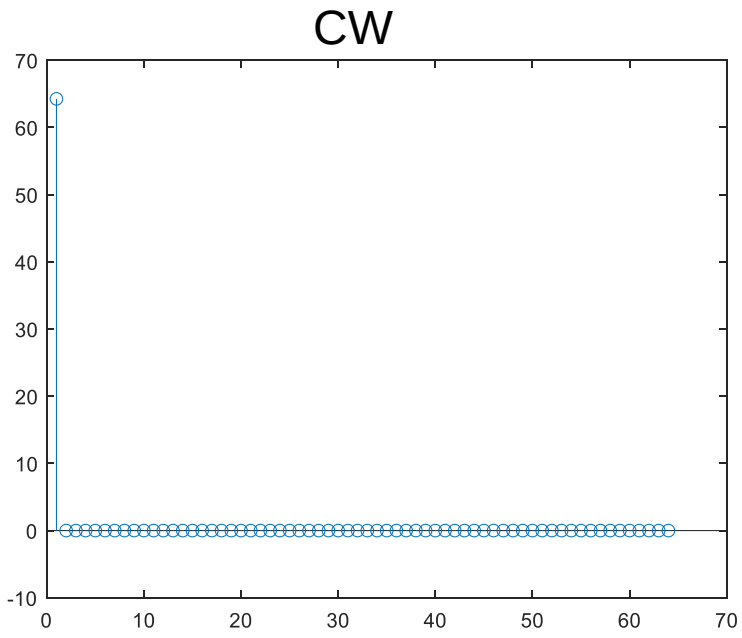
- Let’s put together a sample matrix of a modulated signal without noise.
  - Same parameters
  - Create the signal

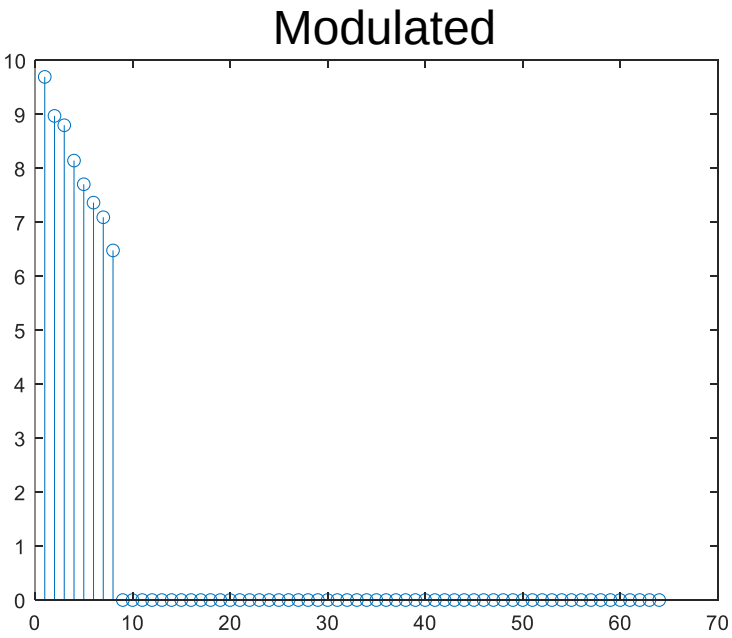
```
samples_per_symbol = 8;
num_symbols = num_samples_in_simulation/samples_per_symbol;
test_signal = exp(1.0i*2*pi*0.125*[0:(num_samples_in_simulation-1)]);
test_signal=test_signal(:);
symbol_vector = randi([0 1], num_symbols, 1)*2-1;
modulation_vector = repelem(symbol_vector,samples_per_symbol);
modulation_vector=modulation_vector(:);

modulated_test_signal = test_signal.*modulation_vector;
```

  - Same reshape

Experiment – Noiseless Modulated (2)

- Now take the eigen values





Experiment – Noisy Signal (1)

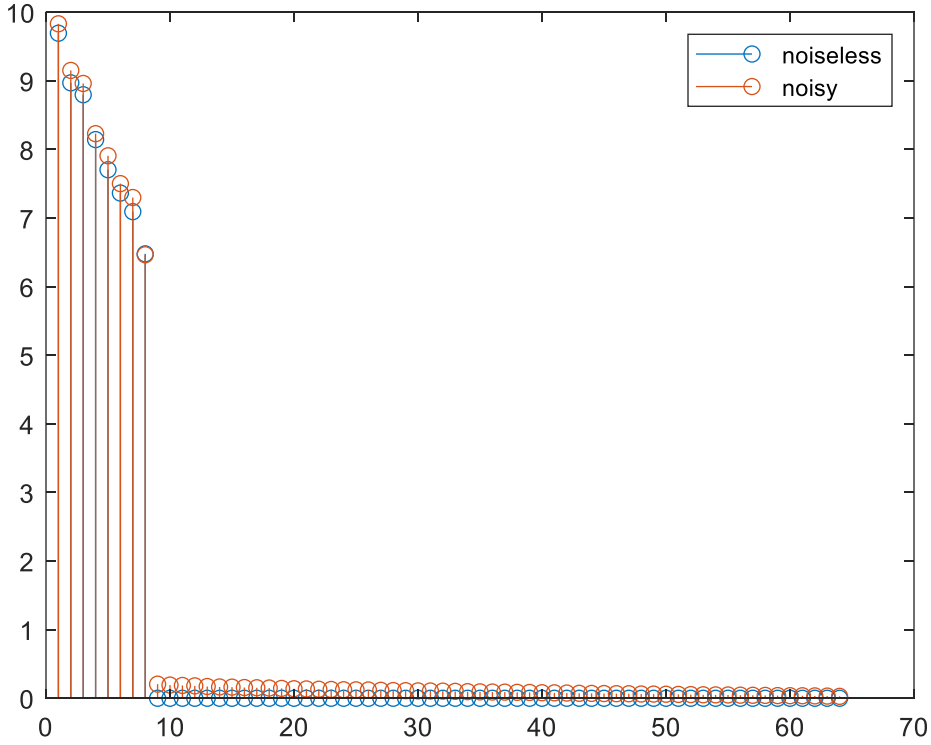
- Now we add noise.
  - Same parameters
  - Same modulated signal
  - Add Noise

```
noise_vec = randn(num_samples_in_simulation,1);
noise_vec = noise_vec + 1.0i*randn(num_samples_in_simulation,1);
noise_vec = noise_vec - mean(noise_vec);
noise_vec = noise_vec/sqrt(var(noise_vec));
noise_vec = noise_vec/sqrt(10);

noisy_modulated_test_signal = modulated_test_signal + noise_vec;
```

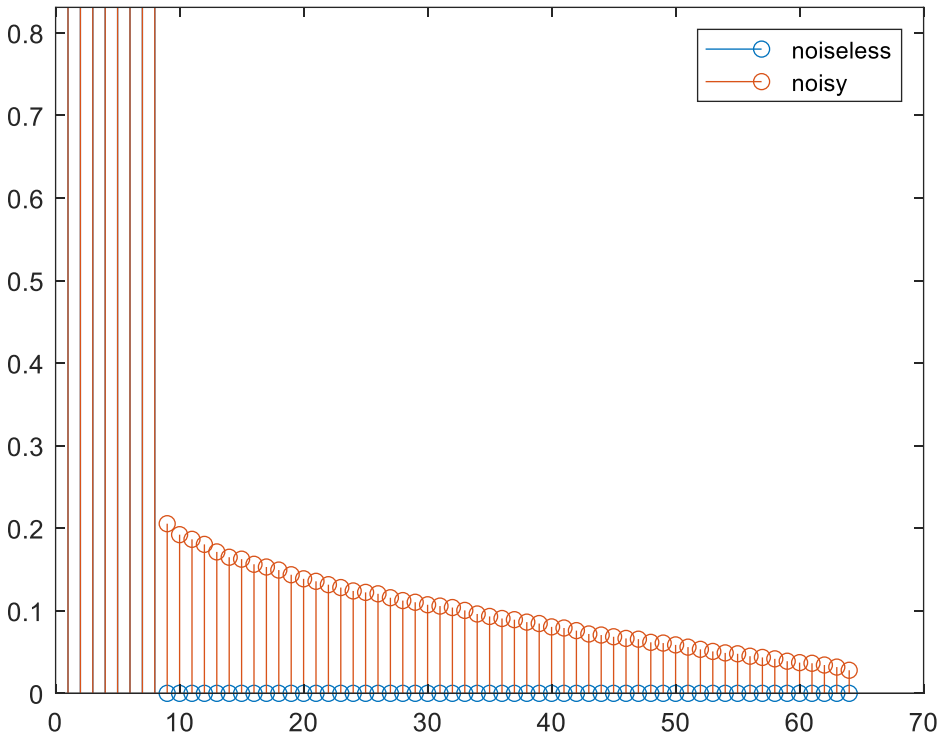
  - Same reshape

Experiment – Noisy Signal (2)

```
eig_vector = eig(sigma_matrix);

sorted_eig_vector = sort(eig_vector, 'descend');

figure
stem(sorted_eig_vector)
```



Experiment – Noisy Signal (3)



Minimum Descriptive Length (1)

- We estimate K using the minimum descriptive length (MDL) criterion
  - $K = \underset{0 \le k < N}{\operatorname{argmin}} \left(-(L-k) N log \frac{\varphi(k)}{\theta(k)} + \frac{1}{2} k(2L-k) log N \right)$

Iterate over all possible k , $k \in (0, N-1)$

- Note that this is what equation 5.20 on page 131 of your text *Wireless Coexistence* should be.

"Detection of Signals by Information Theoretic Criteria", Wax and Kailath, 1988

Minimum Descriptive Length (2)

$$\varphi(k) = \prod_{i=k}^{N-1} \lambda_i^{\frac{1}{N-k}}$$

Geometric mean of $n-k$ smallest eigenvalues

$$\theta(k) = \frac{1}{N-k} \sum_{i=k}^{N-1} \lambda_i$$

Arithmetic mean of $n-k$ smallest eigenvalues

Equations 5.21 and 5.22 in *Wireless Coexistence*

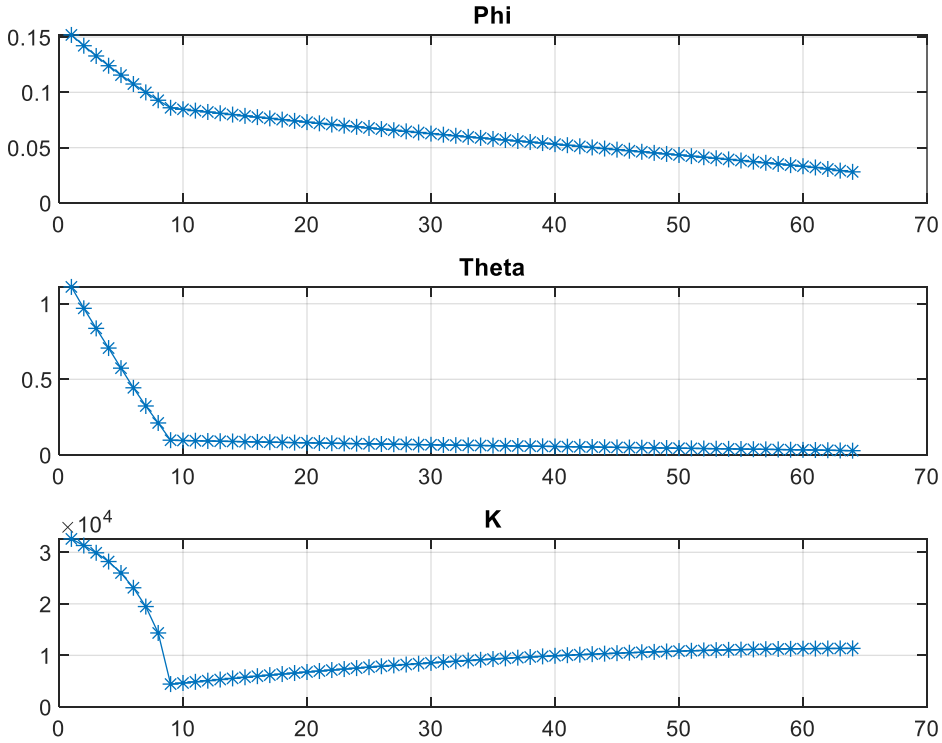
"Detection of Signals by Information Theoretic Criteria", Wax and Kailath, 1988

Estimate the Noise Power

- Call the provided calcKmin function
- Estimate Noise Power
- Calculate estimation error relative to the actual value

```
[kVal, kMin] = calcKMin(sorted_eig_vector, num_observations, num_samples_per_observation);


noise_eigenvalues = sorted_eig_vector(kMin+1:num_samples_per_observation);
estimated_noise_power = mean(noise_eigenvalues);
actual_noise_power = var(noise_vec);


error_val = abs(actual_noise_power-estimated_noise_power)/actual_noise_power
```

Plot from calcKmin



JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2020, All Rights Reserved.